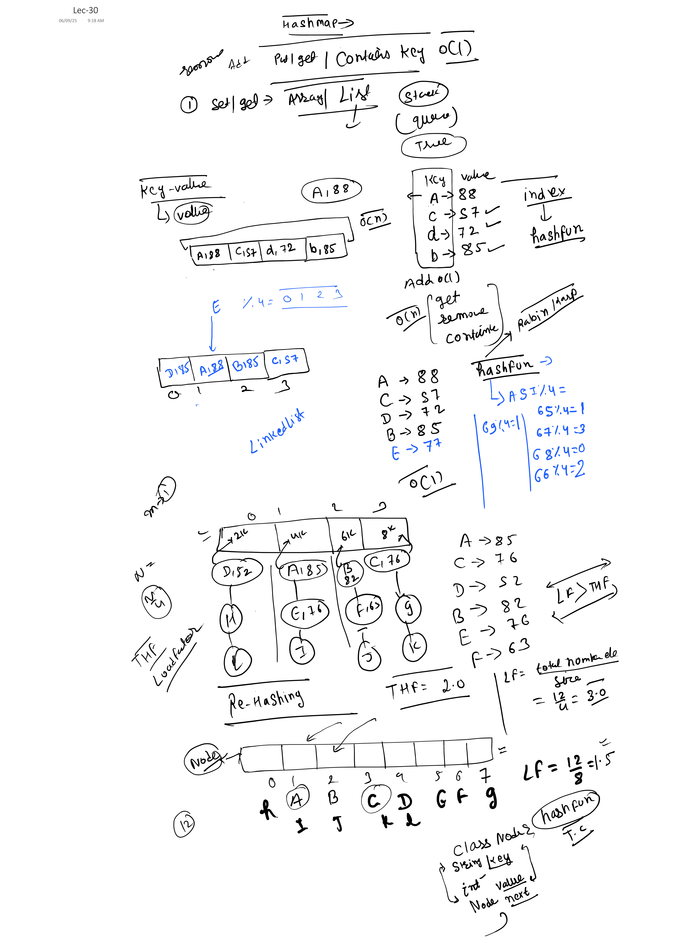
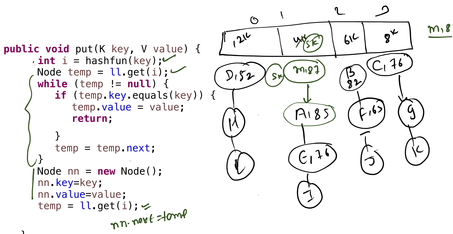
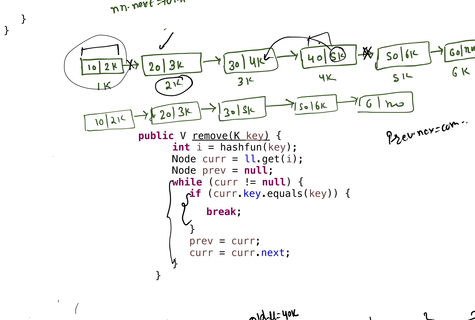
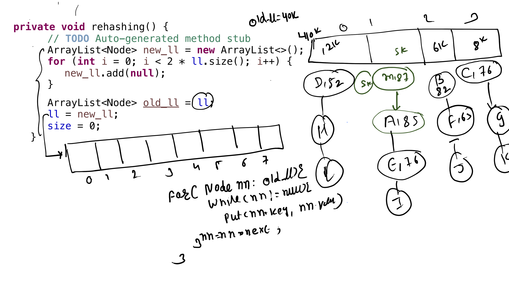




```
public void put(K key, V value) {
    int i = hashfun(key);
    Node temp = ll.get(i);
    while (temp != null) {
        if (temp.key.equals(key)) {
            temp.value = value;
            return;
        }
        temp = temp.next;
    }
    Node nn = new Node();
    nn.key=key;
    nn.value=value;
    temp = ll.get(i);
    nn.next =temp
}
}
```




```
public V remove(K key) {
        int i = hashfun(key);
        Node curr = ll.get(i);
        Node prev = null;
        while (curr != null) {
            if (curr.key.equals(key)) {
                break;
            }
            prev = curr;
            curr = curr.next;
        }
```




```
private void rehashing() {
    // TODO Auto-generated method stub
    ArrayList<Node> new_ll = new ArrayList<>();
    for (int i = 0; i < 2 * ll.size(); i++) {
        new_ll.add(null);
    }
    ArrayList<Node> old_ll = ll;
    ll = new_ll;
    size = 0;
}
```

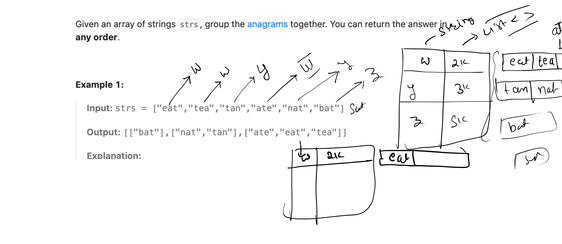
Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

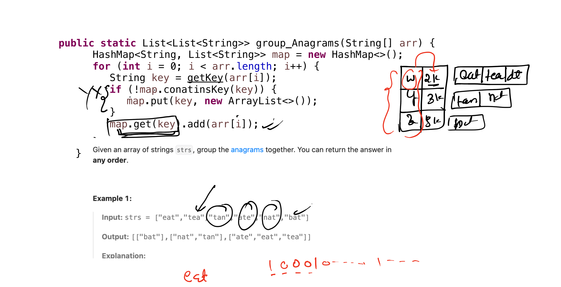
Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:




```
public static List<List<String>> group_Anagrams(String[] arr) {
        HashMap<String, List<String>> map = new HashMap<>();
        for (int i = 0; i < arr.length; i++) {
            String key = getKey(arr[i]);
            if (!map.conatinsKey(key)) {
                map.put(key, new ArrayList<>());
            }
            map.get(key).add(arr[i]);
    }
```



Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation: